# Assignment: Start a New App with the Speckit Starter Kit

**Goal:** Create your own GitHub repository for a **new** product using the OC CS Speckit **starter kit** (empty Vue + Express shells, Cursor rules, and SDD methodology — **no** Todo feature specs or answer-key code). Rename the product, set up local env, then write and implement Feature 1 with Cursor.

**Related docs:** STARTER-KIT.md · features/framework.md · writing-feature-requirements.md · writing-feature-design.md

**Not this assignment:** Rebuilding the Todo example — use ASSIGNMENT-rebuild-todo.md ( `reset:example` ) instead. Reading-only tour of Speckit — use ASSIGNMENT-walkthrough-todo.md.

---

## Learning outcomes

By the end of this assignment you will be able to:

1. Obtain and unpack the Speckit starter kit into a fresh local folder.
2. Create a public GitHub repo and push the starter scaffold.
3. Rename placeholders ( `Speckit App` , `speckit-db` , package names) for your product.
4. Separate **scaffold** ( `main` ) from **integration** ( `dev` ) and **feature** branches.
5. Write a Feature 1 spec and implement it with Cursor against the Definition of Done.

---

## Prerequisites

| Tool | Notes |
|---|---|
| Node.js 24+ | `node -v` |
| npm | Comes with Node |
| MySQL | XAMPP, WAMP, or native MySQL |
| Git + GitHub account | Browser + git CLI |
| Cursor | Agent mode for implementation |
| Starter kit zip | From your instructor, **or** build it (Part B) |

---

## Part A — Create your GitHub repository

1. Open GitHub → New repository.
2. Set:
   - **Repository name:** e.g. `myapp-speckit` or `<project>-speckit-<your-username>` (your product name, not `todo-speckit` )
   - **Visibility:** Public
   - **Do not** add a README, `.gitignore` , or license (keep it empty so the first push is clean)
3. Create the repository. Copy the HTTPS or SSH URL, e.g. `https://github.com/<you>/myapp-speckit.git`

---

## Part B — Obtain the starter kit and load your repo

Start from the **starter kit zip** so you get empty shells and **no** Todo `features/feature-*.md`. Do **not** copy Todo specs into this project.

### B1. Get `speckit-starter-kit.zip`

**Option A — Instructor / course zip (usual)**
Download or copy the `speckit-starter-kit.zip` your course provides.

**Option B — Build the zip yourself** (if you have the full OC CS Speckit / todospeckit tree):

```
# In a separate folder that already contains OC CS Speckit (with scripts/)
npm install
npm run starter:zip
# → dist/speckit-starter-kit.zip
```

See STARTER-KIT.md for details.

### B2. Unzip and open in Cursor

1. Unzip `speckit-starter-kit.zip` into a folder named for your product (e.g. `myapp-speckit`).
2. Open that folder in **Cursor**: File → Open Folder… (macOS and Windows). Optionally, from a system terminal inside the folder run `cursor .` if the Cursor CLI is installed. Work in **IDE / Agent** mode for the rest of this assignment.

### B3. Initialize git and push to GitHub

Open a terminal in Cursor (Terminal → New Terminal). The terminal starts in the project folder, so you do not need to `cd`. Replace `<my-repo>` with the full GitHub URL you copied in Part A step 3:

```
git init
git add .
git commit -m "Initial import: Speckit starter kit"
git branch -M main
git remote add origin <my-repo>
git push -u origin main
```

Confirm on GitHub that your repo has the scaffold (`features/framework.md`, empty `features/reference/`, `.cursor/rules/`, `frontend/`, `backend/`) and **does not** contain `features/feature-1-user-auth.md` … `feature-5-*.md`.

---

## Part C — Rename the product

Make the kit yours before writing features. Use Cursor **Find in Files** (`Cmd+Shift+F` / `Ctrl+Shift+F`) for project-wide replace.

### C1. Package names

| File | Default `name` | Change to (example) |
|---|---|---|
| `package.json` (root) | `speckit-app` | `myapp-speckit` |
| `frontend/package.json` | `speckit-app-frontend` | `myapp-frontend` |
| `backend/package.json` | `speckit-app-backend` | `myapp-backend` |

### C2. Placeholders

| Placeholder | Replace with | Typical hits |
|---|---|---|
| `Speckit App` | Your display name | `README.md`, `Home.vue`, `server.js` log line |
| `speckit-db` | Your MySQL database name (e.g. `myapp-db`) | `.env.example`, `.env.test.example`, `db.config.js` |
| `/api/` | Your API mount path (only if your course requires a different prefix) | `backend/server.js`, `frontend/src/services/services.js` |
| Ports `8082` / `3200` | Only if you must change them | Vite config, Express `PORT`, CORS |

Optional (Agility only): `AGILITY_SCOPE` in `.env.agility.example`, and `DEFAULT_PROJECT` in `scripts/agility/backlog-data.mjs`.

### C3. Commit the rename on `main`

Use **Cursor** Source Control (recommended):

1. Open **Source Control** ( `Ctrl+Shift+G` / `Cmd+Shift+G` ).
2. Confirm you are on `main`.
3. Stage all rename changes, commit e.g. `Rename Speckit placeholders for <Product Name>`, then **Push**.

Alternatively, ask Cursor **Agent**:

```
Commit all product rename changes on main and push to origin main.
```

Or in the terminal:

```
git add -A
git commit -m "Rename Speckit placeholders for my product"
git push origin main
```

---

## Part D — Local setup

**Platform note:** `git` and `npm` commands are the same on macOS and Windows. Where a step copies files or chains `cd` with a command, **macOS / Linux** and **Windows** variants are shown.

### D1. Install dependencies

In the terminal enter these commands:

```
npm install --prefix frontend
npm install --prefix backend
npm install   # root — PDF / tooling scripts (optional)
```

### D2. Create MySQL databases

Use **phpMyAdmin** (e.g. via XAMPP: http://localhost/phpmyadmin) or **MySQL Workbench** to create the two databases named in your `.env.example` after Part C (examples below use `myapp-db` — use **your** names):

```
CREATE DATABASE `myapp-db`;
CREATE DATABASE `myapp-db-test`;
```

In phpMyAdmin: open the SQL tab (or use New → database name) and run the statements.
In MySQL Workbench: open a query tab connected to your local server and run the same statements.

### D3. Configure environment

A `.env` file holds **local secrets and settings** (MySQL host/user/password, database name, JWT secret, port) that the backend reads at startup. These values differ per machine, so they are **not** checked into git — you copy from the checked-in `.env.example` templates instead.

**macOS / Linux**

```
cp backend/.env.example backend/.env
cp backend/.env.test.example backend/.env.test
```

**Windows (Command Prompt or PowerShell)**

```
copy backend\.env.example backend\.env
copy backend\.env.test.example backend\.env.test
```

Edit `backend/.env` and `backend/.env.test` with your MySQL user/password and confirm `DB_NAME` matches the databases you created.
Do **not** commit `.env` files.

### D4. Verify the shell

Harness tests only (no feature product code yet):

```
npm test
```

Open **two** terminals to start the empty app:

**macOS / Linux**

```
# Terminal 1
cd backend && npm run dev

# Terminal 2
cd frontend && npm run dev
```

**Windows (Command Prompt or PowerShell)**

```
REM Terminal 1
cd backend
npm run dev

REM Terminal 2
```

```
cd frontend
npm run dev
```

| Service | URL (defaults) |
|---|---|
| Frontend | http://localhost:8082 |
| API | http://localhost:3200/api/ (starter default; change only if you renamed the mount path) |

---

## Part E — Branch setup ( `main` + `dev` )

Constitution: `main` = starter scaffold only. All feature work merges into `dev`, never into `main`.

### E1. Create `dev` from `main`

Use **Cursor** (recommended):

1. Confirm you are on `main` (status bar).
2. Click the branch name → **Create new branch…** → name it `dev` → create from `main`.
3. **Publish Branch** (or Sync) so `dev` exists on GitHub.

Alternatively, ask Cursor **Agent**:

```
Create a branch named dev from main and push it to origin.
```

Or in the Cursor terminal:

```
git checkout main
git checkout -b dev
git push -u origin dev
```

Optional baseline tag on `main` (terminal):

```
git checkout main
git tag scaffold-v1
git push origin scaffold-v1
```

---

## Part F — Write and implement Feature 1

Your product specs live in **your** `features/feature-*.md` files. Do **not** paste Todo feature specs from the example app.

### F1. Author the Feature 1 spec

1. Read features/framework.md and the student guides:
   - writing-feature-requirements.md
   - writing-feature-design.md
2. Add `features/feature-1-<short-name>.md` using the framework template ( `# Feature: …` , **Status**, **Input**, stories, **FR-00N**, **Assumptions**, **Edge Cases**, **SC-00N**, **Key Entities**, Screen/API/Data as needed, Gherkin, Test Coverage Map, **Agent implementation request**, **Definition of Done**).
3. Add a row to `features/README.md` .

4. Commit the spec on a feature branch (or commit the spec first on `feature/1-…` before implementation — either is fine if the branch is from `dev`).

### F2. Start the feature branch

Use **Cursor** (recommended):

1. Switch to `dev` and sync/pull.
2. **Create new branch...** named `feature/1-<short-name>` (must match the spec **Branch pattern**).
3. Optionally **Publish Branch**.

Alternatively, ask Cursor **Agent**:

```
Switch to dev, pull the latest, and create branch feature/1-<short-name> from dev.
```

Or in the terminal:

```
git checkout dev
git pull origin dev
git checkout -b feature/1-<short-name>
```

### F3. Implement with Cursor

On the feature branch, in **Agent** mode:

```
Implement @features/feature-1-….md per its Agent implementation request and Definition of Done.
```

Or implement layer by layer:

| Step | Ask Cursor for | Verify |
|---|---|---|
| 1 | Backend models + associations | Backend starts (`cd backend` then `npm run dev`) |
| 2 | Routes + controllers + auth helpers | API responds |
| 3 | Backend tests (Gherkin / Test Coverage Map) | `npm run test:backend` |
| 4 | Frontend `*Services.js` + axios client | No axios in views |
| 5 | Views + components | Frontend starts (`cd frontend` then `npm run dev`) |
| 6 | Frontend tests | `npm run test:frontend` |
| 7 | Router / e2e manual check | Browser flow works |

After Feature 1: `npm test` from the repo root must pass. Update `features/reference/api.md`, `data-model.md`, and/or `behavior.md` in the **same** commit/PR when schema or API changes.

### F4. Commit and merge into `dev`

1. In **Source Control**, stage, commit, and push the feature branch.
2. Merge into `dev` via a GitHub **Pull Request** (feature → `dev`), or ask Cursor **Agent**:

```
Commit my Feature 1 work, push this feature branch, then merge it into dev and push origin dev.
Do not merge into main.
```

Or in the terminal:

```
git checkout dev
git merge feature/1-<short-name>
git push origin dev
```

### F5. Definition of Done (Feature 1)

- [ ] Every user story and **FR-00N** has matching code
- [ ] **SC-00N** success criteria are met
- [ ] Every Gherkin scenario has ≥1 automated test
- [ ] `npm test` passes
- [ ] Living reference updated when API/schema/behavior changed
- [ ] Manual demo works in the browser
- [ ] Merged into `dev` (not `main`)

### F6. Do not

- Copy Todo `feature-1` … `feature-5` specs into this repo as if they were your product.
- Implement on `main` or `dev` — use `feature/*` only.
- Commit `.env` or Agility tokens.

---

## Part G — Deliverables

1. **Canvas:** Submit your **GitHub repository link** in the text of the Canvas assignment.
2. **Demo:** Demo your working system to the professor or GA (Feature 1 flows required for your section, plus anything else assigned).

Your repo should show:

| Branch / artifact | Expected |
|---|---|
| `main` | Starter kit + product rename (no feature product code) |
| `dev` | Feature 1 (and later features) merged; `npm test` passes |
| `feature/1-…` / PR | Implementation from your written spec into `dev` (not `main`) |

### Suggested commit milestones

1. `Initial import: Speckit starter kit`
2. `Rename Speckit placeholders for <Product>` on `main`
3. `Create dev from main`
4. Feature 1 spec + implementation merged into `dev`

---

## Quick command cheat sheet

`git` and `npm` commands are the same on **macOS** and **Windows**. Only file-copy and some `cd` chaining differ.

```
# After unzip + open folder in Cursor:
git init
git add .
git commit -m "Initial import: Speckit starter kit"
git branch -M main
git remote add origin <my-repo>
git push -u origin main

# Env (macOS / Linux) — after renaming placeholders
npm install --prefix frontend && npm install --prefix backend
cp backend/.env.example backend/.env
cp backend/.env.test.example backend/.env.test

# Env (Windows) — use these instead of cp:
#   npm install --prefix frontend
#   npm install --prefix backend
#   copy backend\.env.example backend\.env
#   copy backend\.env.test.example backend\.env.test
# CREATE DATABASE myapp-db; CREATE DATABASE myapp-db-test;

git checkout -b dev && git push -u origin dev

# Feature 1
git checkout dev && git checkout -b feature/1-<short-name>
# … write features/feature-1-….md, then Cursor implement …
npm test
git checkout dev && git merge feature/1-<short-name> && git push origin dev
```

---

## Troubleshooting

| Problem | Fix |
|---|---|
| Repo still has Todo `feature-1`…`feature-5` | You used the full Todo tree or `reset:example`. Start over from **`speckit-starter-kit.zip`** |
| Agent implements on `main` or `dev` | Stop; check out `feature/N-…` first |
| Agent invents behavior not in the spec | Refuse; update `features/` first or drop the code |
| Wrong stack / API shape | Cite `@.cursor/rules/` (e.g. `api-conventions`, `ui-style-system`) |
| Tests weakened to “pass” | Require real Gherkin coverage; no `expect(true).toBe(true)` |
| Confused with Todo rebuild | That assignment is ASSIGNMENT-rebuild-todo.md — this one is **starter kit only** |